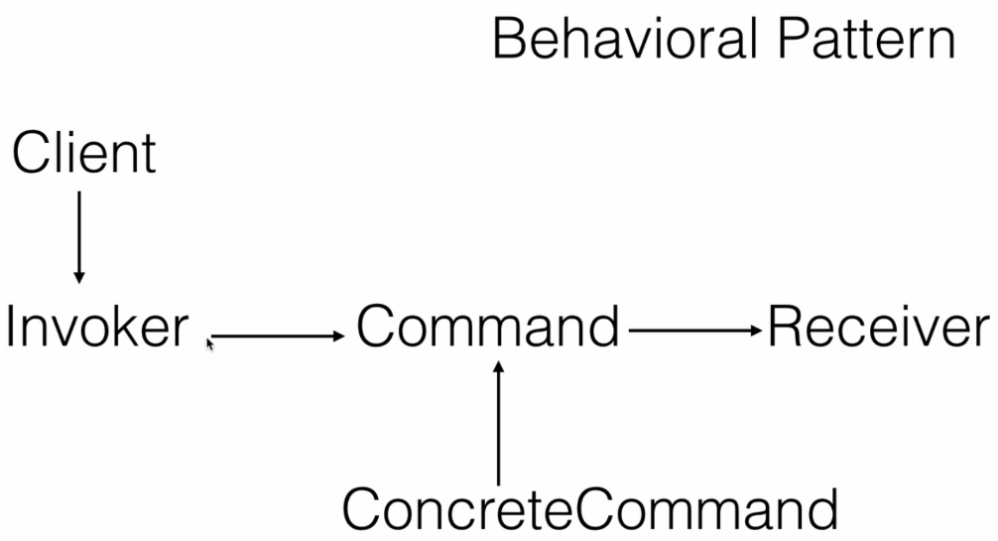
# 07- Command Pattern

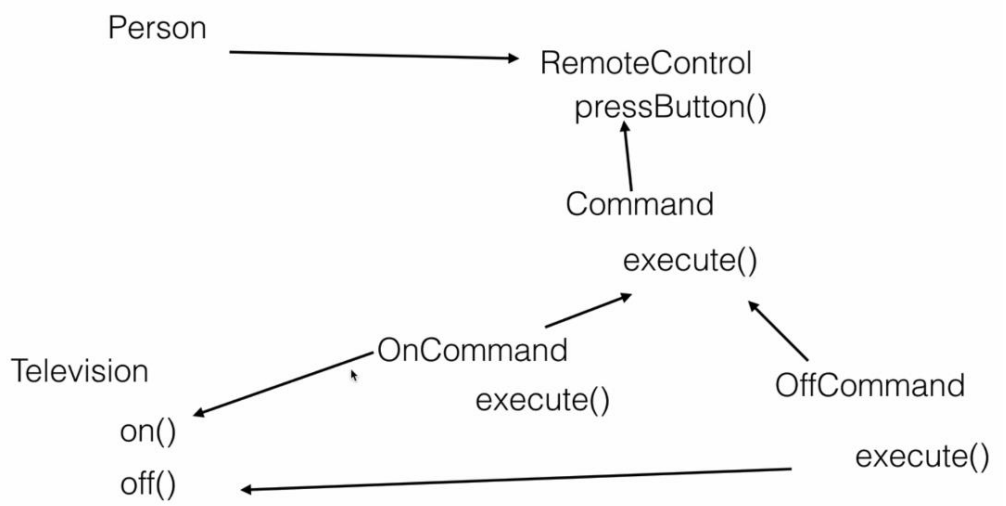
- The Command Pattern is a behavioral design pattern that turns a request into an object.

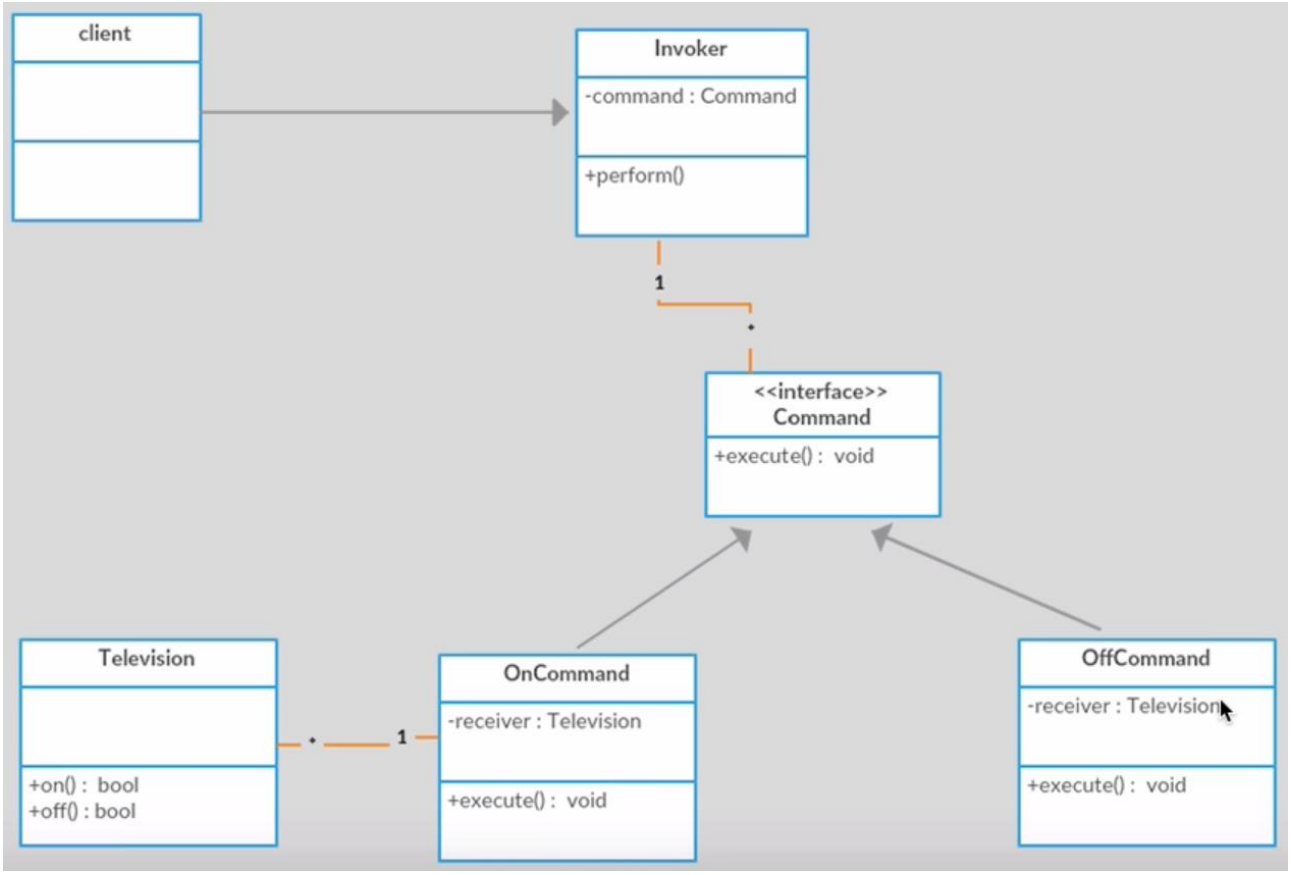



## Core Components

1. **Command (Interface)**
   - Declares `execute()` (and sometimes `undo()`)
2. **ConcreteCommand**
   - Implements the command
   - Calls the receiver's method
3. **Receiver**
   - The actual object that performs the action
4. **Invoker**
   - Triggers the command (e.g., a button)
5. **Client**
   - Creates and wires everything together

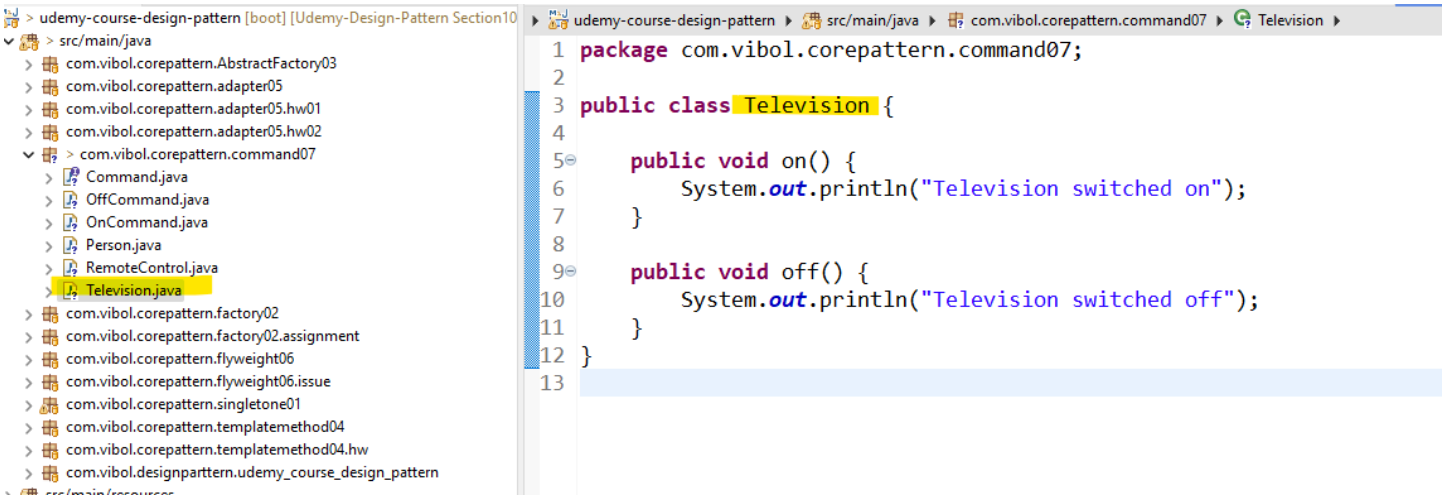
# Example:


Person
RemoteControl
pressButton()
Command
execute()
OnCommand
execute()
OffCommand
execute()
Television
on()
off()


client
Invoker
-command : Command
+perform()
1
*
<<interface>>
Command
+execute() : void
Television
+on() : bool
+off() : bool
*
1
OnCommand
-receiver : Television
+execute() : void
OffCommand
-receiver : Television
+execute() : void

- កន្លែង Television ជាកន្លែងដែលធ្វើការ បិទ ឬ បើក ចឹងយើងត្រូវធ្វើការដាក់ Turn on or Turn off inside it.
- ដើម្បីធ្វើការបង្កើត OnCommand or OffCommand វាត្រូវការ input object Television
- Interface Command គឺជាកន្លែងដែលយើងត្រូវធ្វើការដាក់ Function ដែលយើងចង់អោយ ធ្វើការ execute មានន័យថា subclass: On and Off ត្រូវតែធ្វើប្រតិបត្តិការនឹង
- RemoteControl : យើងគិតដូចជាកន្លែងគេចុត ប៉ូតុងបញ្ជាចឹង គេអាច ចុត បើ ឬ បិទ ចឹងបើគេ ចុត បើក យើង Set វាទៅបើ បើគេចុត បិទ យើង Set វា បិទ។

1- Television




2- Interface

```java
package com.vibol.corepattern.command07;

public interface Command {

    public void execute();
}
```

## 3- On and off command

OffCommand.java

src/main/java ▸ com.vibol.corepattern.command07 ▸ OffCommand ▸

```java
package com.vibol.corepattern.command07;

public class OffCommand implements Command {

    Television television;

    public OffCommand(Television television) {
        this.television = television;
    }

    @Override
    public void execute() {
        television.off();
    }

}
```

OnCommand.java

udemy-course-design-pattern ▸ src/main/java ▸ com.vibol.corepattern.command07 ▸ OnCommand ▸

```java
package com.vibol.corepattern.command07;

public class OnCommand implements Command {

    private Television television;

    public OnCommand(Television television) {
        this.television = television;
    }

    @Override
    public void execute() {
        television.on();
    }

}
```

## 4- RemoteControl

RemoteControl.java

udemy-course-design-pattern ▸ src/main/java ▸ com.vibol.corepattern.command07

```java
package com.vibol.corepattern.command07;

public class RemoteControl {

    private Command command;

    public void pressButton() {
        command.execute();
    }

    public Command getCommand() {
        return command;
    }

    public void setCommand(Command command) {
        this.command = command;
    }
}
```

5- Person

```java
package com.vibol.corepattern.command07;

public class Person {

    public static void main(String[] args) {

        Television television = new Television();

        RemoteControl remoteControl = new RemoteControl();

        OnCommand onCommand = new OnCommand(television);

        remoteControl.setCommand(onCommand);

        remoteControl.pressButton();

        OffCommand offCommand = new OffCommand(television);
        remoteControl.setCommand(offCommand);

        remoteControl.pressButton();

    }

}
```